VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI - 590 018

A Mini Project Report on

"Hospital Lab Test Result Portal: Patient login portal with RSA-encrypted PDF lab reports"

Submitted in partial fulfilment of the course

Cryptography and Network Security (BCS703)

for the award of the Degree of Bachelor of Engineering in

Computer Science and Design

Submitted by

Dhyan V

1AT23CG040

Under the Guidance of

Mrs. Nayanika

Associate Professor, Dept. of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ATRIA INSTITUTE OF TECHNOLOGY

BANGALORE - 560024

2026-27

ATRIA INSTITUTE OF TECHNOLOGY

Department of Computer Science & Engineering

BANGALORE-560024



CERTIFICATE

This is to certify that the mini project entitled **"Hospital Lab Test Result Portal: Patient login portal with RSA-encrypted PDF lab reports"** has been successfully carried out by **Dhyan V** bearing USN **1AT23CG040**, a bonafide student of **Atria Institute of Technology**, in partial fulfilment of the course **Cryptography and Network Security (BCS703)** for the award of the degree of **Bachelor of Engineering in Computer Science and Design** of **Visvesvaraya Technological University**, Belagavi, during 2026-2027. The project report has been approved as it satisfies the academic requirements prescribed for the said course.

Mrs. Nayanika	**Dr. Devi Kannan**	**Dr. Rajesha S**
Associate Professor, CSE	HoD, CSE	Principal
Project Guide		

ATRIA INSTITUTE OF TECHNOLOGY

Department of Computer Science & Engineering

BANGALORE-560024

DECLARATION

I, Dhyan V, bearing the USN: 1AT23CG040, student of Bachelor of Engineering in Computer Science and Design, Atria Institute of Technology, Bangalore, hereby declare that the mini project entitled “Hospital Lab Test Result Portal: Patient login portal with RSA-encrypted PDF lab reports” has been designed and implemented by me under the guidance of Mrs. Nayanika, Associate Professor, Department of Computer Science and Engineering, and is submitted in partial fulfilment of the course Cryptography and Network Security (BCS703) of Visvesvaraya Technological University, Belagavi, during the academic year 2026-27.

I further declare that this work has not been submitted, in part or in full, to any other university or institution for the award of any degree or course.

Dhyan V

1AT23CG040

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without mentioning the people who made it possible. I take this opportunity to express my sincere gratitude to everyone who guided and supported me in completing this mini project.

I extend my heartfelt gratitude to the Trustees, Management and Board Members of Atria Institute of Technology, and to Dr. Rajesha S, Principal, for providing an excellent learning environment and the facilities required to carry out this project.

I am deeply grateful to Dr. Devi Kannan, Head of the Department of Computer Science and Engineering, for her constant encouragement and for giving me the opportunity to work on this project as part of the course Cryptography and Network Security.

I express my sincere thanks to my guide, Mrs. Nayanika, Associate Professor, Department of Computer Science and Engineering, for her valuable guidance, timely suggestions and continuous support at every stage of this project, from understanding the cryptographic concepts to reviewing the final report.

I also thank all the teaching and non-teaching staff of the Department of Computer Science and Engineering for their help and cooperation during the course of this work.

Lastly, I express my heartfelt appreciation to my parents and my friends for their encouragement, patience and moral support, which helped me complete this project successfully.

Dhyan V
1AT23CG040

ABSTRACT

Laboratory test reports contain some of the most sensitive personal data a hospital holds. In many systems they are e-mailed as ordinary PDF attachments or stored unencrypted on a portal server, so a single server breach, a misdirected message or a curious insider can expose every patient's results. A leaked medical record cannot be taken back.

This project, **SecureLab**, is a full-stack Hospital Lab Test Result Portal with a patient login portal and separate laboratory-staff and administrator workspaces. It is designed around one rule: a lab report is never stored in readable form, and the server never holds the key that can read it. Laboratory staff either enter test results, from which the system builds a PDF, or upload an existing PDF. The backend encrypts the PDF with a fresh AES-256-GCM key, wraps that key with the patient's RSA-2048 public key using RSA-OAEP (SHA-256), and signs the original report with the hospital's RSA-PSS key. The patient's key pair is generated inside the browser with the Web Crypto API; the private key is encrypted with a passphrase (PBKDF2-HMAC-SHA-256, 310,000 iterations, then AES-256-GCM) and kept by the patient as a key file that is never sent to the server. When a report is opened, the browser decrypts it, checks the GCM authentication tag, recomputes the SHA-256 digest and verifies the hospital's signature before showing anything.

Around this core the system adds scrypt password hashing, JWT sessions that are rejected immediately when an account is deactivated, TOTP two-factor authentication, server-enforced role-based access control, rate limiting, security response headers, automatic sign-out of idle sessions, and an audit log that administrators can search and export.

The system is implemented with React and the Web Crypto API on the frontend, Python and Flask with the cryptography library on the backend, and SQLite or PostgreSQL for storage. All 9 automated tests pass, covering ciphertext bit-flip detection, wrong-key and forged-signature rejection, cross-patient access (HTTP 403) and the rejection of deactivated sessions. Even a complete copy of the database and the report store reveals no report content. The project demonstrates authenticated encryption, public-key cryptography, digital signatures, key derivation, hashing and multi-factor authentication in a practical healthcare setting.

Keywords: Lab report security, hybrid encryption, AES-GCM, RSA-OAEP, RSA-PSS digital signature, PBKDF2, TOTP, role-based access control.

CONTENTS

CHAPTER 1 7
INTRODUCTION
Problem Statement 7
Objectives 7
Scope 7
CHAPTER 2 8
SYSTEM DESIGN
2.1 System Architecture 8
2.2 Data Flow of a Lab Report 8
2.3 User Roles 9
2.4 Database Design 9
CHAPTER 3 10
CRYPTOGRAPHIC TECHNIQUES USED
3.1 Hybrid Encryption of a Report 10
3.2 Digital Signature and Verification 10
3.3 Patient Key Vault 10
3.4 Authentication and Access Control 11
3.5 Audit Logging 11
CHAPTER 4 12
IMPLEMENTATION
4.1 Technology Stack 12
4.2 Modules 12
4.3 Lab Report Generation 12
4.4 REST API Overview 12
4.5 User Interface 13
CHAPTER 5 14
RESULTS AND SECURITY ANALYSIS
5.1 Test Results 14
5.2 Attack Resistance 14
5.3 Application Screenshots 15
CHAPTER 6 18
CONCLUSION AND FUTURE WORK
Limitations and Future Work 18
REFERENCES 19

CHAPTER 1
INTRODUCTION

Patients increasingly expect to view their laboratory results online. Yet a lab report is a permanent record of a person's health, and unlike a password it cannot be changed after it leaks. Many portals still store reports as plain files, or protect the whole store with a single server key, which means one breach, one stolen backup or one dishonest administrator exposes every patient at once.

PROBLEM STATEMENT

To design a hospital lab result portal in which a patient logs in to view their own reports, and in which every report is stored and transmitted only in encrypted form, can be read only by the patient it belongs to, carries a verifiable proof that the hospital issued it, and in which tampering, unauthorised access and misuse of accounts are prevented or detectable.

OBJECTIVES

- Provide separate patient, laboratory-staff and administrator workspaces with server-enforced role-based access.
- Encrypt every PDF report with AES-256-GCM and wrap its key for the patient using RSA-OAEP, so only the patient's private key can open it.
- Generate each patient's key pair in the browser so that the private key never reaches the server.
- Digitally sign every report with the hospital's RSA-PSS key and verify the signature and SHA-256 digest before a report is opened.
- Protect accounts with scrypt password hashing, JWT sessions, TOTP two-factor authentication, rate limiting and automatic sign-out.
- Let laboratory staff create reports from test templates, with out-of-range values flagged automatically, as well as upload existing PDFs.
- Keep an audit trail that administrators can search and export.

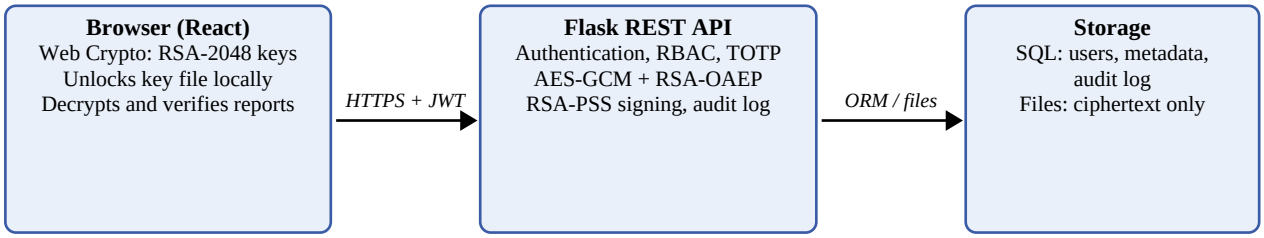
SCOPE

The project covers patient registration and key creation, report creation and upload, encryption and signing, patient-side decryption and verification, two-factor authentication, account management and audit review. It is an academic prototype: it uses synthetic data only, and clinical integration (HL7/FHIR), hardware-protected keys, a public-key infrastructure for the hospital's signing key, and account or key recovery are outside the current scope and are listed as future work.

CHAPTER 2
SYSTEM DESIGN

The system has three tiers: a React browser client that performs all patient-side cryptography, a Flask REST API that authenticates users and performs report encryption and signing, and a storage layer made of a SQL database (SQLite or PostgreSQL) for accounts, metadata and audit events, and a private file store that holds only encrypted report files.

2.1 SYSTEM ARCHITECTURE



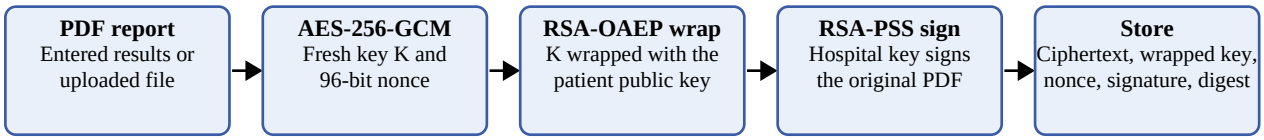
Private keys never leave the browser; the server holds only ciphertext and public keys.

Figure 2.1: System architecture

2.2 DATA FLOW OF A LAB REPORT

1. A patient registers and signs in. In the browser, Web Crypto generates an RSA-2048 (OAEP, SHA-256) key pair.
2. The private key (PKCS#8) is encrypted with a key derived from the patient's passphrase using PBKDF2-HMAC-SHA-256 (310,000 iterations, random 16-byte salt) and AES-256-GCM, and is downloaded as a .securelabkey file. Only the public key is registered with the API.
3. Laboratory staff select the patient and either enter test results (the server builds the PDF in memory) or upload a PDF, whose header and size (at most 12 MB) are checked.
4. The server generates a fresh 256-bit AES key and a 96-bit nonce and encrypts the PDF with AES-256-GCM.
5. The AES key is wrapped with the patient's public key (RSA-OAEP, SHA-256). The original PDF is signed with the hospital's RSA-PSS key and its SHA-256 digest is computed.
6. Only the ciphertext file and the wrapped key, nonce, signature, digest and metadata are stored. The application never writes the plaintext PDF to disk.
7. When the patient requests a report, the API releases the payload only to the owning patient (HTTP 403 for anyone else) and records the access in the audit log.
8. The browser unlocks the key file, unwraps the AES key, decrypts the report (the GCM tag must verify), recomputes SHA-256 and verifies the RSA-PSS signature. The PDF opens only if every check succeeds.

A. Laboratory side (server)



B. Patient side (browser)

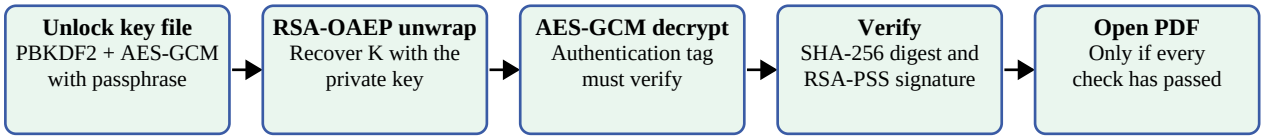


Figure 2.2: Encryption (laboratory side) and decryption (patient side) of a report

2.3 USER ROLES

Role	Permissions
Patient	Register, create the encryption key, enable two-factor authentication, search and sort own reports, decrypt and download own reports, change password, view own activity.
Laboratory staff	Search patients, create reports from test templates, upload PDFs, list, search and delete the reports they sent. Cannot read a report after it is encrypted.
Administrator	Create staff accounts, activate or deactivate any account, view dashboard counters, search and export the audit log, delete reports. Cannot decrypt patient reports.
Visitor (no account)	View the landing page, register as a patient or sign in. All other API endpoints require a valid token.

2.4 DATABASE DESIGN

Table	Main columns
users	public_id (UUID), name, email (unique), password_hash, role (patient / lab / admin), patient_code, public_key_pem, is_active, mfa_secret, mfa_enabled, created_at
reports	id (UUID), patient_id and created_by_id (foreign keys), report_name, test_type, original_filename, encrypted_path, wrapped_aes_key, nonce, signature, sha256_hex, created_at
audit_logs	actor_id, actor_email, action, resource_id, details, ip_address, created_at

CHAPTER 3
CRYPTOGRAPHIC TECHNIQUES USED

The project applies the core concepts of the Cryptography and Network Security course, as mapped below.

Purpose	Technique	CNS Concept
Report encryption	AES-256-GCM, fresh 256-bit key and 96-bit random nonce per report	Symmetric AEAD: confidentiality + integrity
Report key transport	RSA-OAEP (2048-bit patient key, SHA-256, MGF1)	Hybrid public-key cryptosystem
Report authenticity	RSA-PSS (3072-bit hospital key, SHA-256, 32-byte salt)	Digital signature
Report integrity	SHA-256 digest of the original PDF, recomputed by the browser	Cryptographic hash function
Patient key at rest	PBKDF2-HMAC-SHA-256 (310,000 iterations) + AES-256-GCM	Password-based key derivation
Passwords	scrypt with per-password salt (Werkzeug)	Memory-hard password hashing
Sessions	JWT signed with HMAC-SHA-256, 30-minute expiry, account-status check on every request	Token authentication, revocation
Second factor	TOTP (time-based one-time password, 6 digits, 30 s)	Multi-factor authentication
Abuse control	Rate limits: login 5 per minute, registration 5 per hour	Brute-force defence
Transport and browser	HTTPS in deployment, restricted CORS, X-Frame-Options, nosniff, no-store	Transport and web-application security

3.1 HYBRID ENCRYPTION OF A REPORT

A public-key algorithm such as RSA is too slow and too size-limited to encrypt a PDF directly, while a symmetric cipher needs a shared key. SecureLab therefore uses the standard hybrid design. For every report the server draws a new random 256-bit key K and a new 96-bit nonce, and encrypts the PDF with AES-256-GCM, which provides confidentiality and an authentication tag in one operation. K is then encrypted with the patient's RSA public key using OAEP padding with SHA-256. Because K and the nonce are never reused, two encryptions of the same PDF look completely different, and a patient's key opens only that patient's reports.

3.2 DIGITAL SIGNATURE AND VERIFICATION

Encryption alone does not prove who produced a report. The hospital therefore signs the original PDF with its RSA-PSS key (SHA-256, 32-byte salt) and publishes the matching public key through the API. After decryption the browser recomputes SHA-256 and verifies the signature. If the content was altered, or the signature was made by any key other than the hospital's, verification fails and the report is not opened.

3.3 PATIENT KEY VAULT

The patient's private key is exported as PKCS#8 and encrypted in the browser. A 256-bit AES key is derived from the passphrase with PBKDF2-HMAC-SHA-256 (310,000 iterations and a random 16-byte salt) and used with AES-256-GCM and a random 96-bit IV. The resulting JSON key file records the

format, version, KDF, iteration count, salt, IV and encrypted key. It is useless without the passphrase, and the server never receives it. The unlocked key is held only in browser memory and is discarded on sign-out, page reload or idle timeout.

3.4 AUTHENTICATION AND ACCESS CONTROL

- Passwords are stored only as scrypt hashes. Patient passwords need at least 10 characters and staff passwords at least 12, each with letters and digits.
- After login the API issues a JWT (30-minute lifetime). On every request the server confirms that the account still exists and is active, so deactivating an account ends its sessions at once.
- Roles are checked on the server for every endpoint. A report payload is released only to the patient who owns it; laboratory staff and administrators receive HTTP 403.
- TOTP two-factor authentication can be enabled by any user; the login then requires a valid 6-digit code in addition to the password.
- The browser signs the user out automatically after 10 idle minutes, with a 60-second warning.

3.5 AUDIT LOGGING

Registrations, logins and failures, key registration, MFA changes, password changes, report uploads and deletions, report access, and account activation or deactivation are written to the audit_logs table together with the actor and IP address. Administrators can search, filter and export the log as CSV; exported cells that begin with formula characters are escaped so a spreadsheet cannot execute them.

CHAPTER 4
IMPLEMENTATION

4.1 TECHNOLOGY STACK

Layer	Technology	Key Files
Frontend	React 18, Vite 6, Web Crypto API, lucide-react icons, plain CSS	main.jsx, crypto.js, api.js, Landing.jsx, styles.css
Backend	Python 3.13, Flask 3.1, Flask-JWT-Extended, Flask-Limiter, Flask-CORS, cryptography, pyotp, reportlab	app/routes/*.py, services/crypto_service.py, services/report_pdf.py
Database	SQLite (default) or PostgreSQL through SQLAlchemy; encrypted files on disk	models.py, report_storage.py
Testing	pytest: 9 automated tests	tests/test_crypto_service.py, tests/test_api_flow.py

4.2 MODULES

- **Patient workspace:** key generation and unlocking, report library with search and sort, decrypt-and-verify, verified PDF download, account and activity page.
- **Laboratory workspace:** create report from templates with live preview, upload PDF, uploaded-reports list with statistics, search and delete.
- **Administrator workspace:** overview counters, all-users page with activate or deactivate, staff creation, security audit with search, filter and CSV export.
- **Account module (all roles):** change password with strength meter, two-factor setup, recent activity, idle auto sign-out.
- **Landing page:** public introduction to the portal with sign-in and registration entry points.

4.3 LAB REPORT GENERATION

Besides uploading a PDF, staff can create a report directly. They choose the patient and a test category (Complete Blood Count, Lipid Profile, Blood Glucose, Liver Function, Kidney Function, Thyroid Profile, Urine Analysis or Other). The form is pre-filled with the usual parameters, units and reference ranges, all of which can be edited. As results are typed, each value is classified as LOW, HIGH or NORMAL by comparing it with the numeric reference limits, and a live preview shows the report as it will appear. On submission the server validates every field, builds a formatted PDF in memory with the reportlab library, and passes it through the same encrypt, wrap and sign pipeline as an uploaded file.

4.4 REST API OVERVIEW

Method	Endpoint	Access	Purpose
POST	/api/auth/register, /api/auth/login	Public	Patient registration; password + optional TOTP login
POST	/api/auth/change-password	Authenticated	Change own password
POST	/api/auth/mfa/setup, /mfa/enable	Authenticated	Start and confirm TOTP

Method	Endpoint	Access	Purpose
POST	/api/keys/register	Patient	Register RSA public key
GET	/api/keys/hospital-public	Public	Hospital signature verification key
POST	/api/reports/upload	Lab, Admin	Encrypt and store an uploaded PDF
POST	/api/reports/generate	Lab, Admin	Build a PDF from results, then encrypt and store it
GET	/api/reports	Authenticated	Role-appropriate report metadata
GET	/api/reports/{id}/payload	Owning patient	Ciphertext and cryptographic metadata
DELETE	/api/reports/{id}	Lab (own), Admin	Delete a report and its encrypted file
PATCH	/api/admin/users/{id}/active	Admin	Activate or deactivate an account
GET	/api/admin/audit-logs	Admin	Review the audit trail

4.5 USER INTERFACE

The interface is a single-page React application. Each role has its own accent colour (teal for patients, blue for laboratory staff and indigo for administrators), a pinned sidebar, and a responsive layout that also works on phones. The patient dashboard uses an animated healthcare-themed background, and the landing page includes scroll and hover animations that are switched off automatically for users who request reduced motion. The JWT is kept in memory rather than in browser storage, so a page reload signs the user out.

CHAPTER 5
RESULTS AND SECURITY ANALYSIS

All 9 automated tests pass with pytest. They exercise the cryptographic service directly and the REST API end to end against an in-memory database.

5.1 TEST RESULTS

No.	Test	What it proves	Result
1	Hybrid round trip	AES-GCM and RSA-OAEP recover the original PDF; the SHA-256 digest matches and the RSA-PSS signature verifies	Pass
2	Ciphertext bit-flip	Changing a single bit of the ciphertext makes AES-GCM decryption fail (InvalidTag)	Pass
3	Wrong patient key	A different RSA private key cannot unwrap the AES key	Pass
4	Forged or altered report	Altered content, or a signature by another key, fails RSA-PSS verification	Pass
5	Fresh key and nonce	Encrypting the same PDF twice gives a different nonce, wrapped key and ciphertext	Pass
6	Weak RSA key	A 1024-bit patient public key is rejected	Pass
7	Report access control	Lab upload returns 201; the owner gets the payload (200); another patient gets 403	Pass
8	Password, deactivation, delete	Wrong current password is refused; the old password stops working; a deactivated user's token and login return 401	Pass
9	Report generation	LOW/HIGH/NORMAL rule and input validation; missing key returns 409; patients cannot generate (403); the patient can fetch the result	Pass

5.2 ATTACK RESISTANCE

Attack	Defence	Result	Verified by
Database or file store stolen	Reports are AES-256-GCM ciphertext; the AES key is wrapped for the patient; no private key is on the server	Only unreadable data obtained	Design; tests 1 and 3
Ciphertext modified	GCM authentication tag	Decryption refused	Automated test 2
Report forged or altered	RSA-PSS signature and SHA-256 digest	Report not opened	Automated test 4
Another patient requests a report	Ownership check on the payload endpoint	HTTP 403	Automated test 7
Staff or admin reads a report	Payload endpoint is restricted to the owning patient	HTTP 403	Code review
Session of a deactivated account	Account status checked on every request	HTTP 401	Automated test 8
Weak or substituted keys	Minimum 2048-bit RSA; OAEP unwrap needs the matching private key	Rejected	Automated tests 3, 6

Attack	Defence	Result	Verified by
Password guessing	scrypt hashing; login limited to 5 attempts per minute	Slowed and blocked	Configuration
Stolen password only	TOTP second factor required at login	Login refused without code	Manual demonstration
Clickjacking, MIME sniffing, caching	X-Frame-Options DENY, nosniff, Cache-Control no-store	Blocked	Response headers
Unattended signed-in browser	10-minute idle sign-out; key locked again	Session cleared	Manual demonstration

5.3 APPLICATION SCREENSHOTS

The following figures show the main screens of the running application.

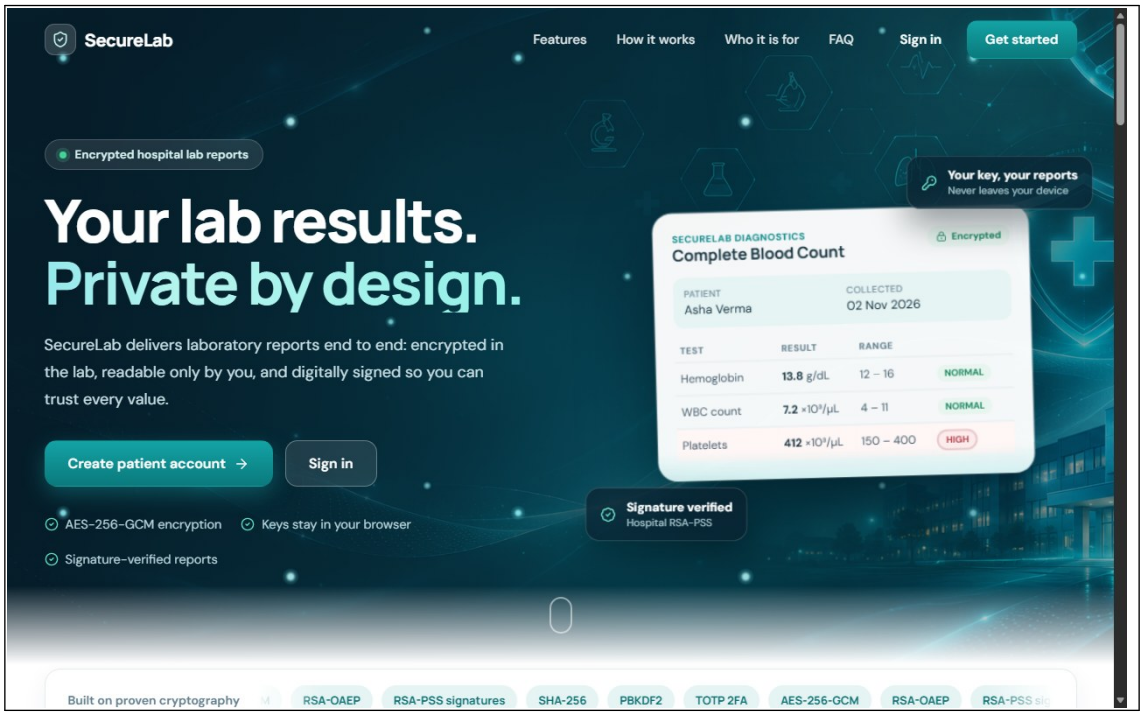


Figure 5.1: Landing page

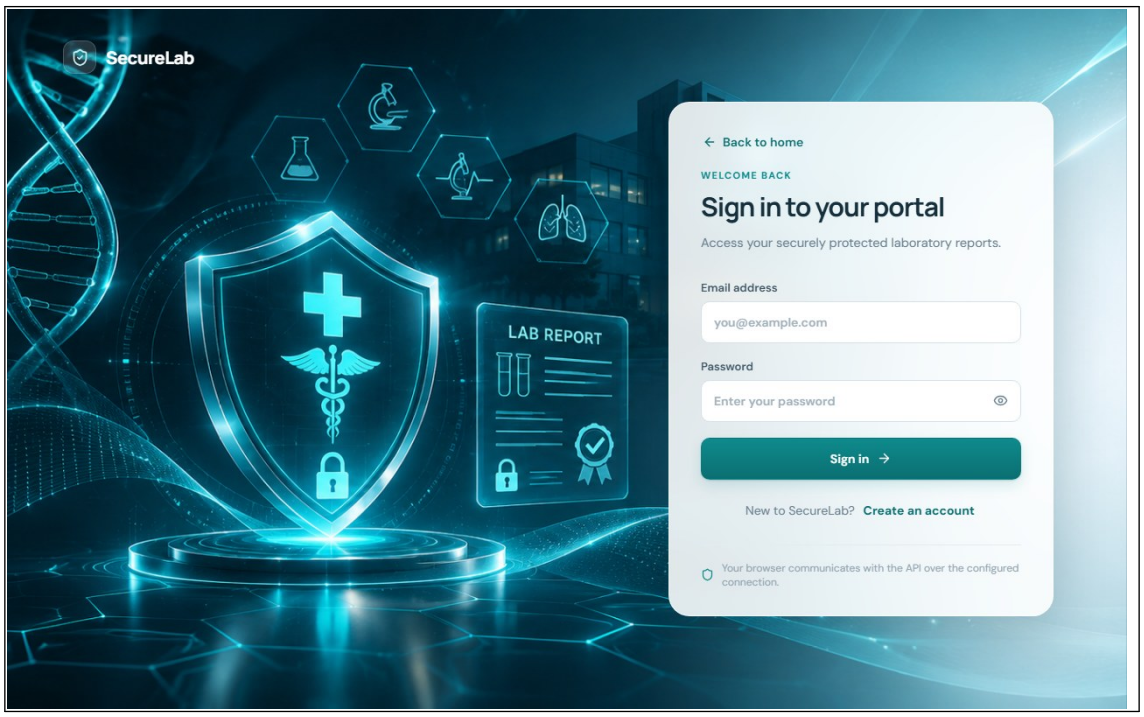


Figure 5.2: Sign-in page

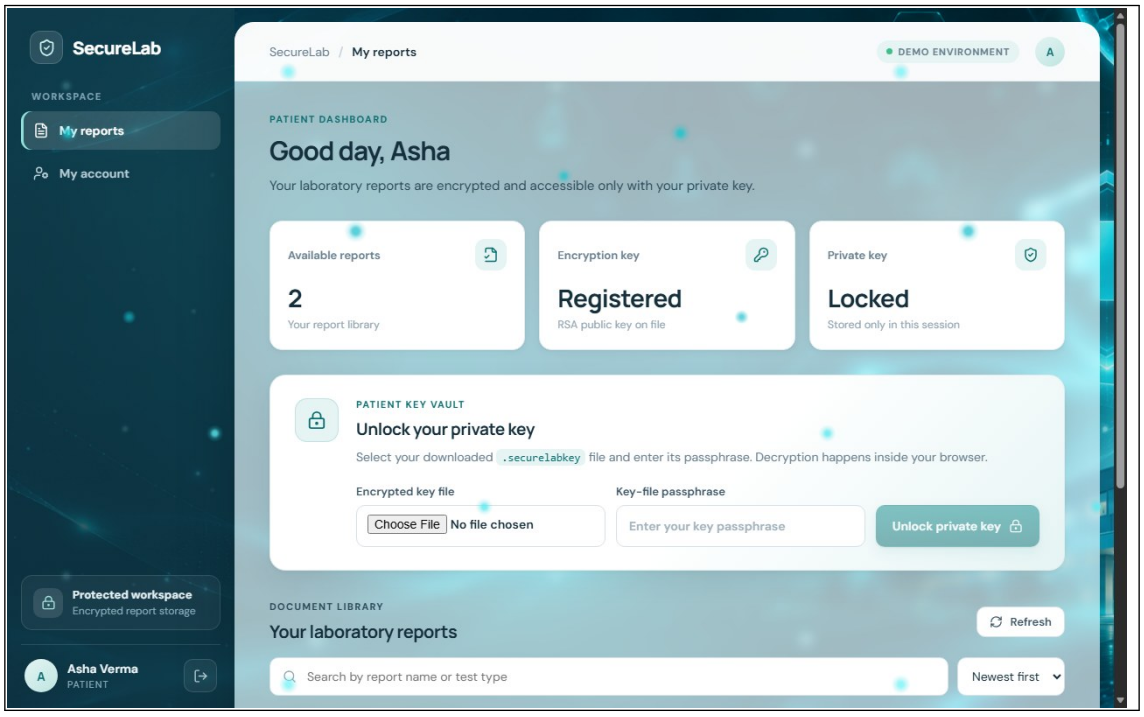


Figure 5.3: Patient dashboard with key vault and report library

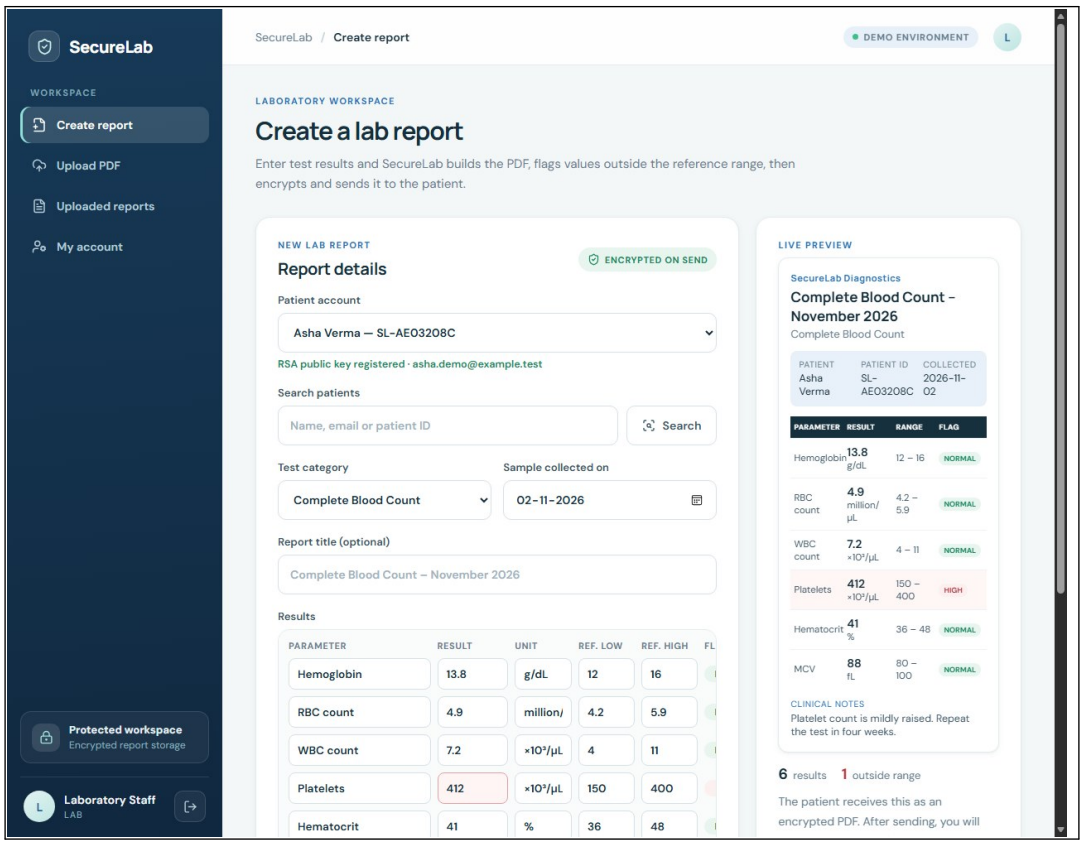


Figure 5.4: Laboratory staff creating a report, with live preview and automatic flagging of the raised platelet count

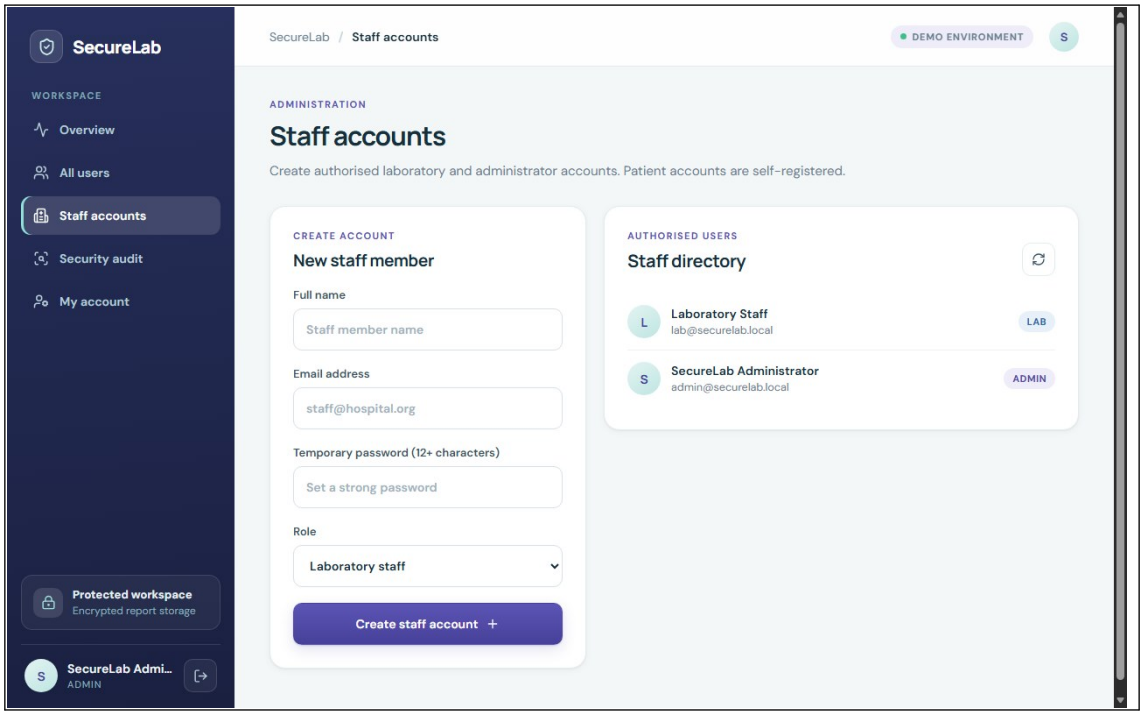


Figure 5.5: Administrator workspace, staff accounts

CHAPTER 6
CONCLUSION AND FUTURE WORK

SecureLab shows that standard cryptographic primitives (AES-GCM, RSA-OAEP, RSA-PSS, SHA-256, PBKDF2 and scrypt), combined through hybrid encryption and patient-held keys, can keep laboratory reports confidential at rest and in transit while still giving patients a simple login portal. Because the patient's private key is created and used only in the browser, a complete leak of the database and the report store exposes no readable report, and every tampering, forgery and cross-patient access attempt in the test suite was detected or refused.

The project gave me practical experience of applying CNS concepts such as authenticated encryption, public-key cryptography, digital signatures, password-based key derivation, multi-factor authentication and access control to a real full-stack system.

LIMITATIONS AND FUTURE WORK

- The hospital signing key is stored unprotected on the server's disk for demonstration. It should be moved to an HSM or cloud key-management service, with rotation and a certificate-based trust chain for the hospital's public key.
- TOTP secrets are stored in the database in plain form and should be encrypted at rest. Password reset and account recovery are not implemented.
- If a patient loses the key file or passphrase, the reports cannot be recovered. A reviewed key-escrow or recovery design, and patient key rotation, are future work.
- Rate limiting uses in-memory storage and should move to a shared store such as Redis for multi-server deployments. Audit logs are not tamper-proof and should be forwarded to append-only central logging.
- Encryption does not currently bind the report identifier as additional authenticated data; adding it would also stop ciphertexts being swapped between database rows.
- Add clinician delegation and consent, emergency access, HL7/FHIR integration, malware scanning of uploads, and a formal security review before any real-world use.

REFERENCES

1 W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed., Pearson.

2 NIST SP 800-38D, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*.

3 RFC 8017, *PKCS #1: RSA Cryptography Specifications Version 2.2* (RSA-OAEP and RSA-PSS).

4 RFC 8018, *PKCS #5: Password-Based Cryptography Specification Version 2.1* (PBKDF2).

5 RFC 7914, *The scrypt Password-Based Key Derivation Function*.

6 RFC 7519, *JSON Web Token (JWT)*.

7 RFC 6238, *TOTP: Time-Based One-Time Password Algorithm*.

8 NIST FIPS 180-4, *Secure Hash Standard (SHS)*.

9 W3C, *Web Cryptography API*, Recommendation.

10 Project source code: https://github.com/Gududhyan/Secure_lab